

CRANK Developer Onboarding Guide

You own the build, the ticket, and the documentation -- in that order, by yourself.

THE ROLE IN ONE SENTENCE

Developers own the full lifecycle of Run phase output: building it, ticketing it, documenting it, and demoing it when required. The biggest shift from traditional delivery is documentation -- you create the Jira ticket that describes what you built, not the other way around. Nobody hands you a pre-written ticket to execute.

WHERE YOU SHOW UP IN THE FLYWHEEL

Phase	Your role
Charge	Consulted -- you may be pulled in to sanity-check a signal, but the CRANK Lead makes the actionability call.
Run	Responsible -- this is your phase. Build against the Charge signal and ship before it feels polished.
Assess	Responsible -- when output does not perform, you name the failure out loud, same session.
Navigate	Consulted -- the CRANK Lead calls direction; you inform it with what you learned in Assess.
Dev KICK	Responsible -- this is your meeting. Blockers, velocity, and direction all get confirmed here.
Stakeholder KICK	Consulted -- you attend only when a live demo is warranted. No proxy presenters.

THE SINGLE BIGGEST HABIT CHANGE

In a BA- or PM-led model, someone else writes the ticket and hands it to you. In CRANK, you write your own Jira ticket and documentation, created from your working understanding as you build -- not dictated upstream and not written after the fact from memory. This is the rule teams slide back on fastest under pressure. It does not relax under deadline pressure -- it matters most then, because you are the only accurate source of what actually got built.

Rule of thumb: if you could not hand your ticket to a stranger and have them understand what shipped and why, it is not done yet.

RUNNING RUN AND ASSESS

- A working artifact exists by mid-cycle -- not just a plan.
- Ship to a real environment, not a staging sandbox. Staging does not count toward ship rate.

- When something breaks or underperforms, stop and name it explicitly: "we assumed X, it was actually Y."
- State the lesson as an action -- "next cycle we will Z" -- within 20 minutes. Assess is a diagnosis, not a debrief.

WHAT YOU OWE DEV KICK

Dev KICK runs on a fixed, scheduled cadence tied to the cycle itself. Come ready to confirm priorities still hold, review your own cycle metrics honestly, name every blocker you are carrying out loud, and ask anything you are unsure about. The meeting closes when the room is aligned, not by the clock -- and before it closes, you should be able to repeat the current direction back in one sentence without checking with anyone.

COMMON DEVELOPER MISTAKES

- Waiting for a written requirements doc before starting -- Charge does not produce one; you build from the signal as captured.
- Letting someone else (a BA, PM, or the CRANK Lead) write your ticket -- even with good intentions, this reverts the team to the old model.
- Shipping only to staging and calling it done -- ship rate counts real environments only.
- Skipping Assess because the output was "mostly fine" -- name what was learned anyway.
- Showing up to Stakeholder KICK to present when there is nothing new to demo -- that block gets skipped, not filled with a status update.

SUCCESS INDICATORS

- Your tickets are written from what you actually built, not a spec someone handed you.
- You can name a failure out loud the same session it happens, without it feeling like an admission.
- You know the current direction cold and don't need to check the steering document to repeat it.
- You've never had a Stakeholder KICK demo where the audience saw something they weren't expecting.